Programming of Siemens Simatic PLCs
Basic Operations

- During the program execution, **a PLC calculates a status bit** called **Result of Logic Operations (RLO)**
- RLO is used by the programmer who has to take into account that instructions may
  - **Depend** on RLO
  - **Influence** RLO
  - **Terminate** the calculation of the current RLO

```
U M0.0
U M0.1
= M1.0
O M2.0
O M2.1
= M3.0
```

The AND Instruction **U** depends on the RLO and Influences the RLO

The **assignment** Instruction **=** depends on the RLO, **does not influence** the RLO, **terminates the calculation** of the RLO

The calculation of a **new RLO starts here**

The OR instruction O has the **same behavior** of AND with respect to the RLO

NOTE: RLO is often indicated as **RLC (Risultato Logico Combinatorio)** in the examples given in this course

Assignment, Set and Reset Operations

- Assignment (**=**), Set (**S**) and Reset (**R**) are **bit operations** that **depend on RLO**, **terminate** its evaluation but **do not influence** it
- The **Assignment** operation **copies** the value of RLO on the addressed bit
- The **Set** operation **puts the addressed bit equal to 1**, **if RLO = 1**. If RLO = 0, the instruction **has no effect** on the addressed bit
- The **Reset** operation **puts the addressed bit equal to 0 if RLO = 1.** If RLO = 0, the instruction has no effect on the addressed bit

```
U M0.0
U M0.1
= M1.0
```

The value of RLO, i.e. the results of the AND operation between M 0.0 and M 0.1, is continuously (cycle by cycle) copied on M 1.0

```
U M0.0
U M0.1
S M1.0
```

The first time that RLO = 1, M1.0 is set to 1. Then, M1.0 does not change anymore, whatever being the RLO value

```
U M0.0
U M0.1
R M1.0
```

The first time that RLO = 1, M1.0 is reset to 0. Then, M1.0 does not change anymore, whatever being the RLO value

Load and Transfer Operations/1

- The PLCs we use have **two accumulator registers called ACCU1 and ACCU2**
- ACCU1 and ACCU2 are **32-bit registers** used to **store and handle** variables
- The operation to copy a **variable to ACCU1 is Load (L)**
- The operation to copy the **content of ACCU1 on a variable is Transfer (T)**
- ACCU2 **can not be directly accessed** via instructions
- L and T are operations that use **Byte, Words and Double Words as operands**, **but NOT bits**

L MB20 ←
T AB100 ←

The value of MB20 is copied to ACCU1
The value of ACCU1 (MB20) is copied to AB100.
IMPORTANT NOTE:
with this instruction **(T AB100)** we are **sending commands to the plant**

INSTRUCTION	ACCU1	ACCU2
L MW100	[MW100]	[?]
L MW50	[MW50]	[MW100]
T MW26	[MW50]	[MW100]
L MB10	[MB10]	[MW50]

With the Load instruction, the content of ACCU1 is moved to ACCU2. The former content of ACCU2 is discarded.
The Transfer operation does not change the content of ACCU1 and ACCU2

Load and Transfer Operations/2

INSTRUCTION	ACCU1	ACCU2
L ED16	[ED16]	[?]
T AD28	[ED16]	[?]
T MD0	[ED16]	[?]
L MB10	[MB10]	[ED16]
L AW10	[AW10]	[MB10]
L MD16	[MD20]	[AW10]

Load a 32-bit variable (ED16)
Transfer on a 32-bit variable (AD28)
Transfer ED16 also on MD0

ED16 is moved to ACCU2
MB10 is moved to ACCU2, ED16 is discarded from ACCU2
AW10 is moved to ACCU2, MB10 is discarded from ACCU2

- Load and Transfer operations **do not have any relationship with RLO**
- In other words, **Load and Transfer**
  - **Do not depend** on RLO
  - **Do not influence** RLO
  - **Do not terminate** the current RLO calculation

Comparison Operations/1

- Comparison operations apply to the **contents of ACCU1 and ACCU2**
- Specifically, they compare the content of ACCU2 with that of ACCU1
- Comparisons can be carried out on **either** 8/16 bit numbers or 32 bit numbers
- Comparison operators are:
  - **==** (ACCU2=ACCU1) **<>** (ACCU2≠ACCU1)
  - **<** (ACCU2<ACCU1) **>** (ACCU2>ACCU1)
  - **<=** (ACCU2<=ACCU1) **>=** (ACCU2>=ACCU1)
  - The result of comparison operations **is copied to RLO**

INSTRUCTION	ACCU1	ACCU2	RLO
L MW0	10	[?]	?
L MW2	3	10	?
> I	3	10	1
S M100.0	3	10	1

In this example, MW0 = 10, MW2 = 3
The comparison operation determines RLO =1, which sets M100.0 to 1

Let's suppose this code fragment is run cyclically in the PLC, with unknown values of MW0 and MW2. Finding at a certain time M100.0=1 means that at least in one cycle we had MW0 > MW2

The suffix **I** after the comparison instruction **>** means that the comparison has to be carried out on the **16 low-bits of the accumulator registers**
The suffix **D** means that the comparison is made on the **whole 32 bits of the accumulators**
The suffix **R** means that the comparison is made on **floating point numbers**

Comparison Operations/2

INSTRUCTION	ACCU1	ACCU2	RLO
L MD20	100	[?]	?
L MD4	3	100	?
>D	3	100	1
U E0.0	3	100	1
= M0.0	3	100	1

- Comparison operations **do not depend on RLO**
- Comparison operations **influence RLO**
- Comparison operations **do not terminate the calculation of the current RLO**

In this example, MD20 = 100, MD4 = 3
The result of the comparison is combined in a AND operation with E0.0. It is supposed that E0.0=1. Thus, the AND operation has result 1. Such value is then assigned to M0.0

INSTRUCTION	ACCU1	ACCU2	RLO
O E0.0	[?]	[?]	1
O E0.1	[?]	[?]	1
L MB30	10	[?]	1
L MB24	4	10	1
<=I	4	10	0
= M2.3	4	10	0

In this example it is supposed MD30 = 10, MB24 = 4, E0.0 = 1, E0.1 = 1.
The result of the comparison **changes the RLO value**. Such value is then assigned to M2.3

NOTE: this is **not an example of good programming**, because the OR operations **should be followed by an instruction that terminates** the RLO elaboration